

# ProjectManagerApp — Hướng Dẫn 05: JavaFX Controllers

---

**Package:** `com.projectmanager` (App) · `com.projectmanager.ui` (SceneSwitcher) · `com.projectmanager.ui.controllers`

---

## App.java (Entry Point JavaFX)

```
package com.projectmanager;

import com.projectmanager.ui.SceneSwitcher;
import java.io.IOException;
import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class App extends Application {

    @Override
    public void start(Stage primaryStage) throws IOException {
        SceneSwitcher.setStage(primaryStage);
        primaryStage.setTitle("Project Manager");
        primaryStage.setResizable(false);

        // Mở màn hình login khi khởi động
        FXMLLoader loader = new FXMLLoader(
            App.class.getResource("/com/projectmanager/ui/views/login.fxml"));
        primaryStage.setScene(new Scene(loader.load()));
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

---

## SceneSwitcher.java

```
package com.projectmanager.ui;

import com.projectmanager.App;
import java.io.IOException;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
```

```

import javafx.stage.Stage;

// Utility: chuyển đổi giữa các màn hình FXML – tất cả controller dùng class này
public class SceneSwitcher {

    private static Stage stage;

    public static void setStage(Stage s) { stage = s; }

    public static void switchScene(String fxml) throws IOException {
        FXMLLoader loader = new FXMLLoader(
            App.class.getResource("/com/projectmanager/ui/views/" + fxml));
        stage.setScene(new Scene(loader.load()));
    }

    // Overload: trả về controller sau khi load – dùng khi cần truyền data sang
    // màn mới
    public static <C> C switchSceneAndGetController(String fxml) throws
    IOException {
        FXMLLoader loader = new FXMLLoader(
            App.class.getResource("/com/projectmanager/ui/views/" + fxml));
        stage.setScene(new Scene(loader.load()));
        return loader.getController();
    }
}

```

## LoginController.java

```

package com.projectmanager.ui.controllers;

import com.projectmanager.models.dto.LoginRequest;
import com.projectmanager.models.entity.User;
import com.projectmanager.service.AuthService;
import com.projectmanager.session.UserSession;
import com.projectmanager.ui.SceneSwitcher;
import java.io.IOException;
import java.net.URL;
import java.util.ResourceBundle;
import javafx.fxml.FXML;
import javafx.fxml.Initializable;
import javafx.scene.control.*;

public class LoginController implements Initializable {

    @FXML private TextField      txtUsername;
    @FXML private PasswordField txtPassword;
    @FXML private Label          lblMessage;

    private final AuthService authService = new AuthService();
}

```

```

@Override
public void initialize(URL url, ResourceBundle rb) {
    lblMessage.setText("");
}

@FXML
private void handleLogin() throws IOException {
    try {
        User u = authService.login(new LoginRequest(
            txtUsername.getText().trim(),
            txtPassword.getText()
        ));

        if (u == null) { showMsg("Sai username hoac mat khau.", false);
return; }
        if (!u.status) { showMsg("Tai khoan bi khoa. Lien he admin.", false);
return; }

        UserSession.set(u);
        SceneSwitcher.switchScene("dashboard.fxml");

    } catch (IllegalArgumentException e) {
        showMsg(e.getMessage(), false);
    } catch (Exception e) {
        showMsg("Loi he thong: " + e.getMessage(), false);
    }
}

@FXML
private void goRegister() throws IOException {
    SceneSwitcher.switchScene("register.fxml");
}

private void showMsg(String msg, boolean success) {
    lblMessage.setStyle("-fx-text-fill: " + (success ? "#A6E3A1" : "#F38BA8")
+ ";";
        + "-fx-font-size: 12px; -fx-font-style: italic;");
    lblMessage.setText(msg);
}
}

```

## RegisterController.java

```

package com.projectmanager.ui.controllers;

import com.projectmanager.service.AuthService;
import com.projectmanager.ui.SceneSwitcher;
import java.io.IOException;
import java.net.URL;
import java.util.ResourceBundle;

```

```
import javafx.fxml.FXML;
import javafx.fxml.Initializable;
import javafx.scene.control.*;

public class RegisterController implements Initializable {

    @FXML private TextField      txtUsername;
    @FXML private PasswordField txtPassword;
    @FXML private PasswordField txtConfirm;
    @FXML private TextField      txtEmail;
    @FXML private ComboBox<String> cbRole;
    @FXML private Label          lblMessage;

    private final AuthService authService = new AuthService();

    @Override
    public void initialize(URL url, ResourceBundle rb) {
        cbRole.getItems().addAll("user", "admin");
        cbRole.getSelectionModel().selectFirst(); // "user" mặc định
        lblMessage.setText("");
    }

    @FXML
    private void handleRegister() {
        try {
            boolean ok = authService.register(
                txtUsername.getText().trim(),
                txtPassword.getText(),
                txtConfirm.getText(),
                txtEmail.getText().trim(),
                cbRole.getValue()
            );
            if (ok) {
                showMsg("Tao tai khoan thanh cong. Vui long dang nhap.", true);
                clearForm();
            } else {
                showMsg("Dang ky that bai. Kiem tra ket noi DB.", false);
            }
        } catch (IllegalArgumentException e) {
            showMsg(e.getMessage(), false);
        }
    }

    @FXML
    private void goLogin() throws IOException {
        SceneSwitcher.switchScene("login.fxml");
    }

    private void clearForm() {
        txtUsername.clear(); txtPassword.clear();
        txtConfirm.clear(); txtEmail.clear();
        cbRole.getSelectionModel().selectFirst();
    }
}
```

```

        private void showMsg(String msg, boolean success) {
            lblMessage.setStyle("-fx-text-fill: " + (success ? "#A6E3A1" : "#F38BA8")
+ ";";
            + "-fx-font-size: 12px; -fx-font-style: italic;");
            lblMessage.setText(msg);
        }
    }
}

```

## DashboardController.java

```

package com.projectmanager.ui.controllers;

import com.projectmanager.service.ProjectService;
import com.projectmanager.session.UserSession;
import com.projectmanager.ui.SceneSwitcher;
import java.io.IOException;
import java.net.URL;
import java.util.Map;
import java.util.ResourceBundle;
import javafx.fxml.FXML;
import javafx.fxml.Initializable;
import javafx.scene.control.*;

public class DashboardController implements Initializable {

    @FXML private Label lblWelcome;
    @FXML private Label lblRole;
    @FXML private Label lblStats;
    @FXML private Button btnManageUsers; // chỉ hiện với admin

    private final ProjectService<com.projectmanager.models.Task> service =
        ProjectService.getInstance();

    @Override
    public void initialize(URL url, ResourceBundle rb) {
        lblWelcome.setText("Xin chào, " + UserSession.get().username);
        lblRole.setText("Role: " + UserSession.get().role.toUpperCase());
        lblStats.setText("Dang tai du lieu...");

        // Ẩn "Manage Users" với user thường – chỉ admin thấy
        btnManageUsers.setVisible(UserSession.isAdmin());
        btnManageUsers.setManaged(UserSession.isAdmin());

        // Load DB async – không block UI thread, callback cập nhật stats khi xong
        service.loadFromDBAsync(() -> {
            Map<String, Integer> byStatus = service.countByStatus();
            Map<String, Integer> byPriority = service.countByPriority();
            lblStats.setText(String.format(
                "Tong: %d task | Bug: %d Feature: %d | Effort: %dh%n"
                + "Todo: %d | Dang lam: %d | Xong: %d",

```

```

        service.getSize(), service.countBugs(), service.countFeatures(),
service.totalEffort(),
        byStatus.getDefault("todo", 0),
        byStatus.getDefault("in_progress", 0),
        byStatus.getDefault("done", 0)
    ));
    });
}

@FXML private void goAddTask() throws IOException {
    SceneSwitcher.switchScene("add_task.fxml"); }
@FXML private void goTaskList() throws IOException {
    SceneSwitcher.switchScene("task_list.fxml"); }

@FXML
private void goManageUsers() throws IOException {
    // Double-check quyền – dù button đã bị ẩn nhưng vẫn guard ở đây
    if (!UserSession.isAdmin()) return;
    SceneSwitcher.switchScene("user_list.fxml");
}

@FXML
private void handleLogout() throws IOException {
    UserSession.clear();
    ProjectService.reset(); // xóa cache – tránh data của user này rò sang
user tiếp theo
    SceneSwitcher.switchScene("login.fxml");
}
}

```

## AddTaskController.java

```

package com.projectmanager.ui.controllers;

import com.projectmanager.factory.TaskFactory;
import com.projectmanager.models.*;
import com.projectmanager.service.ProjectService;
import com.projectmanager.ui.SceneSwitcher;
import com.projectmanager.utils.Validator;
import java.io.IOException;
import java.net.URL;
import java.util.ResourceBundle;
import javafx.fxml.FXML;
import javafx.fxml.Initializable;
import javafx.scene.control.*;
import javafx.scene.layout.HBox;

public class AddTaskController implements Initializable {

    @FXML private TextField      txtId;

```

```

@FXML private TextField      txtTitle;
@FXML private ComboBox<String> cbPriority;
@FXML private ComboBox<String> cbStatus;
@FXML private ToggleButton    btnBug;
@FXML private ToggleButton    btnFeature;
// Bug fields
@FXML private HBox            rowSeverity;
@FXML private ComboBox<String> cbSeverity;
// Feature fields
@FXML private HBox            rowHours;
@FXML private HBox            rowDeveloper;
@FXML private TextField        txtHours;
@FXML private TextField        txtDeveloper;
@FXML private Label            lblMessage;

private final ToggleGroup typeGroup = new
ToggleGroup();
private final ProjectService<Task> service = ProjectService.getInstance();

@Override
public void initialize(URL url, ResourceBundle rb) {
    cbPriority.getItems().addAll("HIGH", "MEDIUM", "LOW");
    cbPriority.getSelectionModel().selectFirst();

    cbStatus.getItems().addAll("todo", "in_progress", "done");
    cbStatus.getSelectionModel().selectFirst();

    cbSeverity.getItems().addAll("LOW", "MEDIUM", "HIGH", "CRITICAL");
    cbSeverity.getSelectionModel().selectFirst();

    btnBug.setToggleGroup(typeGroup);
    btnFeature.setToggleGroup(typeGroup);
    btnBug.setSelected(true); // Bug mặc định

    // Bug fields hiện, Feature fields ẩn ban đầu
    setFeatureFieldsVisible(false);

    // Listener: đổi loại task → ẩn/hiện fields tương ứng
    typeGroup.selectedToggleProperty().addListener((obs, old, val) -> {
        boolean isBug = (val == btnBug);
        setFeatureFieldsVisible(!isBug);
        setVisible(rowSeverity, isBug);
    });

    lblMessage.setText("");
}

@FXML
private void handleAdd() {
    try {
        // Validate các trường chung
        Validator.requireNonBlank(txtId.getText(), "Task ID");
        Validator.requireNonBlank(txtTitle.getText(), "Tieu de");
    }
}

```

```

        boolean isBug = (btnBug.isSelected());
        String typeCode = isBug ? "B" : "F";

        // Factory Pattern: tạo Task rỗng đúng loại, sau đó set fields
        Task task = TaskFactory.create(typeCode);
        task.id      = txtId.getText().trim().toUpperCase();
        task.title   = txtTitle.getText().trim();
        task.priority = cbPriority.getValue();
        task.status  = cbStatus.getValue();

        if (isBug) {
            Bug bug = (Bug) task;
            bug.severity = cbSeverity.getValue();
        } else {
            Feature feat = (Feature) task;
            feat.estimatedHours =
Validator.parsePositiveInt(txtHours.getText(), "Số giờ");
            String dev = txtDeveloper.getText().trim();
            feat.assign(dev.isEmpty() ? null : dev);
        }

        boolean ok = service.Add(task); // Add dùng finally để log
        if (ok) {
            showMsg("Đã thêm " + (isBug ? "Bug" : "Feature") + ": " + task.id,
true);
            clearForm();
        } else {
            showMsg("Thêm task thất bại. ID có thể đã tồn tại.", false);
        }

    } catch (IllegalArgumentException e) {
        showMsg(e.getMessage(), false);
    }
}

private void setFeatureFieldsVisible(boolean visible) {
    setVisible(rowHours, visible);
    setVisible(rowDeveloper, visible);
}

// setManaged(false) khi ẩn – tránh row chiếm không gian layout khi invisible
private void setVisible(HBox row, boolean visible) {
    row.setVisible(visible);
    row.setManaged(visible);
}

private void clearForm() {
    txtId.clear(); txtTitle.clear(); txtHours.clear(); txtDeveloper.clear();
    cbPriority.getSelectionModel().selectFirst();
    cbStatus.getSelectionModel().selectFirst();
    cbSeverity.getSelectionModel().selectFirst();
    btnBug.setSelected(true);
}

```

```

        private void showMsg(String msg, boolean success) {
            lblMessage.setStyle("-fx-text-fill: " + (success ? "#A6E3A1" : "#F38BA8")
+ ";")
            + "-fx-font-size: 12px; -fx-font-style: italic;");
            lblMessage.setText(msg);
        }

        @FXML
        private void goBack() throws IOException {
            SceneSwitcher.switchScene("dashboard.fxml");
        }

```

## TaskListController.java

```

package com.projectmanager.ui.controllers;

import com.projectmanager.models.*;
import com.projectmanager.service.ProjectService;
import com.projectmanager.session.UserSession;
import com.projectmanager.ui.SceneSwitcher;
import java.io.IOException;
import java.net.URL;
import java.util.List;
import java.util.ResourceBundle;
import javafx.beans.property.*;
import javafx.collections.FXCollections;
import javafx.fxml.FXML;
import javafx.fxml.Initializable;
import javafx.scene.control.*;

public class TaskListController implements Initializable {

    @FXML private TableView<Task> tableView;
    @FXML private TableColumn<Task, String> colType;
    @FXML private TableColumn<Task, String> colId;
    @FXML private TableColumn<Task, String> colTitle;
    @FXML private TableColumn<Task, String> colPriority;
    @FXML private TableColumn<Task, String> colStatus;
    @FXML private TableColumn<Task, Integer> colEffort;
    @FXML private TableColumn<Task, String> colExtra;
    @FXML private ComboBox<String> cbFilterStatus;
    @FXML private ComboBox<String> cbFilterType;
    @FXML private ComboBox<String> cbNewStatus; // đổi status task
    được chọn
    @FXML private Button btnDelete; // chỉ admin thấy
    @FXML private Label lblStatus;
    @FXML private Label lblEffort;

    private final ProjectService<Task> service = ProjectService.getInstance();

```

```

@Override
public void initialize(URL url, ResourceBundle rb) {
    setupColumns();
    setupFilters();

    // Role-based: ẩn Delete với user thường – chỉ admin mới delete được
    btnDelete.setVisible(UserSession.isAdmin());
    btnDelete.setManaged(UserSession.isAdmin());

    loadData(service.getAll());
}

private void setupColumns() {
    // Cột Type: màu đỏ cho Bug, xanh teal cho Feature
    colType.setCellValueFactory(d ->
        new SimpleStringProperty(d.getValue() instanceof Bug ? "BUG" :
"FEAT"));
    colType.setCellFactory(col -> new TableCell<>() {
        @Override
        protected void updateItem(String item, boolean empty) {
            super.updateItem(item, empty);
            if (empty || item == null) { setText(null); setStyle(""); return;
}

            setText(item);
            setStyle("-fx-text-fill: " + ("BUG".equals(item) ? "#F38BA8" :
"#89DCEB")
                + "; -fx-font-weight: bold;");
        }
    });

    colId.setCellValueFactory(d -> new SimpleStringProperty(d.getValue().id));
    colTitle.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().title));

    // Cột Priority: HIGH=đỏ, MEDIUM=cam, LOW=xanh lá
    colPriority.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().priority));
    colPriority.setCellFactory(col -> new TableCell<>() {
        @Override
        protected void updateItem(String item, boolean empty) {
            super.updateItem(item, empty);
            if (empty || item == null) { setText(null); setStyle(""); return;
}

            setText(item);
            String color = switch (item) {
                case "HIGH" -> "#F38BA8";
                case "MEDIUM" -> "#FAB387";
                default -> "#A6E3A1";
            };
            setStyle("-fx-text-fill: " + color + "; -fx-font-weight: bold;");
        }
    });

    // Cột Status: todo=xám, in_progress=cam, done=xanh lá

```

```

        colStatus.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().status));
        colStatus.setCellFactory(col -> new TableCell<>() {
            @Override
            protected void updateItem(String item, boolean empty) {
                super.updateItem(item, empty);
                if (empty || item == null) { setText(null); setStyle(""); return;
}

                setText(item);
                String color = switch (item) {
                    case "in_progress" -> "#FAB387";
                    case "done"         -> "#A6E3A1";
                    default              -> "#6C7086";
                };
                setStyle("-fx-text-fill: " + color + "");
            }
        });

        colEffort.setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().GetEffort()).asObject());
        colEffort.setCellFactory(col -> new TableCell<>() {
            @Override
            protected void updateItem(Integer item, boolean empty) {
                super.updateItem(item, empty);
                setText(empty || item == null ? null : item + "h");
            }
        });

        // Cột Extra: severity cho Bug, developer cho Feature
        colExtra.setCellValueFactory(d -> {
            Task t = d.getValue();
            if (t instanceof Bug)      return new SimpleStringProperty("Sev: " +
((Bug) t).severity);
            if (t instanceof Feature) {
                Feature f = (Feature) t;
                return new SimpleStringProperty(f.isAssigned() ? "Dev: " +
f.getAssignedTo() : "(Chua assign)");
            }
            return new SimpleStringProperty("");
        });
    }

    private void setupFilters() {
        cbFilterStatus.getItems().addAll("(Tat ca)", "todo", "in_progress",
"done");
        cbFilterStatus.getSelectionModel().selectFirst();

        cbFilterType.getItems().addAll("(Tat ca)", "B", "F");
        cbFilterType.getSelectionModel().selectFirst();

        cbNewStatus.getItems().addAll("todo", "in_progress", "done");
        cbNewStatus.getSelectionModel().selectFirst();
    }

```

```

private void loadData(List<Task> data) {
    tableView.setItems(FXCollections.observableArrayList(data));
    int totalEffort = data.stream().mapToInt(Task::GetEffort).sum();
    lblEffort.setText("Tong effort: " + totalEffort + "h | So task: " +
data.size());
    lblStatus.setText("");
}

// — Handlers —————

@FXML
private void handleFilter() {
    String status = cbFilterStatus.getValue();
    String type = cbFilterType.getValue();

    List<Task> filtered = service.getAll();

    // Lọc theo status nếu không phải "(Tat ca)"
    if (!"(Tat ca)".equals(status))
        filtered = service.FilterByStatus(status);

    // Lọc tiếp theo type
    if (!"(Tat ca)".equals(type)) {
        final String t = type;
        filtered = filtered.stream()
            .filter(task -> task.getTypeCode().equals(t))
            .collect(java.util.stream.Collectors.toList());
    }

    loadData(filtered);
    lblStatus.setText("Dang hien thi: " + filtered.size() + " task.");
}

@FXML
private void handleClearFilter() {
    cbFilterStatus.getSelectionModel().selectFirst();
    cbFilterType.getSelectionModel().selectFirst();
    loadData(service.getAll());
}

// Cập nhật status của task được chọn – cả user và admin đều dùng được
@FXML
private void handleUpdateStatus() {
    Task selected = tableView.getSelectionModel().getSelectedItem();
    if (selected == null) { lblStatus.setText("Vui long chon 1 task.");
return; }

    String newStatus = cbNewStatus.getValue();
    if (service.updateStatus(selected.id, newStatus)) {
        loadData(service.getAll()); // refresh table
        lblStatus.setText("Da cap nhat status: " + selected.id + " → " +
newStatus);
    } else {
        lblStatus.setText("Cap nhat that bai.");
    }
}

```

```

    }
}

// Xóa task – CHỈ admin (btnDelete đã bị ẩn với user thường)
@FXML
private void handleDelete() {
    if (!UserSession.isAdmin()) return;    // guard: không thể bị gọi từ UI
    nhưng vẫn guard

    Task selected = tableView.getSelectionModel().getSelectedItem();
    if (selected == null) { lblStatus.setText("Vui long chon task can xoa.");
return; }

    Alert alert = new Alert(Alert.AlertType.CONFIRMATION,
        "Xoa \"" + selected.title + "\"?", ButtonType.YES, ButtonType.NO);
    alert.setTitle("Xac nhan xoa");
    alert.setHeaderText(null);
    alert.showAndWait().ifPresent(btn -> {
        if (btn == ButtonType.YES) {
            if (service.delete(selected.id)) {
                loadData(service.getAll());
                lblStatus.setText("Da xoa: " + selected.id);
            } else {
                lblStatus.setText("Xoa that bai.");
            }
        }
    });
}

@FXML
private void handleReload() {
    lblStatus.setText("Dang tai lai tu DB...");
    service.loadFromDBAsync(() -> {
        loadData(service.getAll());
        lblStatus.setText("Da tai lai du lieu.");
    });
}

@FXML
private void goBack() throws IOException {
    SceneSwitcher.switchScene("dashboard.fxml"); }
}

```

---

## UserListController.java — Admin Only

```

package com.projectmanager.ui.controllers;

import com.projectmanager.models.entity.User;
import com.projectmanager.repository.UserRepository;
import com.projectmanager.session.UserSession;

```

```

import com.projectmanager.ui.SceneSwitcher;
import java.io.IOException;
import java.net.URL;
import java.util.List;
import java.util.ResourceBundle;
import javafx.beans.property.*;
import javafx.collections.FXCollections;
import javafx.fxml.FXML;
import javafx.fxml.Initializable;
import javafx.scene.control.*;

public class UserListController implements Initializable {

    @FXML private TableView<User>          tableView;
    @FXML private TableColumn<User, Integer> colId;
    @FXML private TableColumn<User, String> colUsername;
    @FXML private TableColumn<User, String> colEmail;
    @FXML private TableColumn<User, String> colRole;
    @FXML private TableColumn<User, String> colStatus;
    @FXML private Label                    lblMessage;

    private final UserRepository userRepo = new UserRepository();

    @Override
    public void initialize(URL url, ResourceBundle rb) {
        // Guard: nếu không phải admin → quay về dashboard
        if (!UserSession.isAdmin()) {
            try { SceneSwitcher.switchScene("dashboard.fxml"); }
            catch (IOException e) { e.printStackTrace(); }
            return;
        }

        colId.setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().id).asObject());
        colUsername.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().username));
        colEmail.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().email));

        // Cột Role: admin=xanh dương, user=xám
        colRole.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().role));
        colRole.setCellFactory(col -> new TableCell<>() {
            @Override
            protected void updateItem(String item, boolean empty) {
                super.updateItem(item, empty);
                if (empty || item == null) { setText(null); setStyle(""); return; }

                setText(item.toUpperCase());
                setStyle("admin".equals(item)
                    ? "-fx-text-fill: #89B4FA; -fx-font-weight: bold;"
                    : "-fx-text-fill: #6C7086;");
            }
        });
    }
}

```

```

// Cột Status: active=xanh lá, blocked=đỏ
colStatus.setCellValueFactory(d ->
    new SimpleStringProperty(d.getValue().status ? "Active" : "Blocked"));
colStatus.setCellFactory(col -> new TableCell<>() {
    @Override
    protected void updateItem(String item, boolean empty) {
        super.updateItem(item, empty);
        if (empty || item == null) { setText(null); setStyle(""); return;

        setText(item);
        setStyle("Active".equals(item)
            ? "-fx-text-fill: #A6E3A1; -fx-font-weight: bold;"
            : "-fx-text-fill: #F38BA8; -fx-font-weight: bold;");
        }
    });

loadData();
lblMessage.setText("");
}

private void loadData() {
    List<User> users = userRepo.findAll();
    tableView.setItems(FXCollections.observableArrayList(users));
}

// Toggle block/unblock user được chọn
@FXML
private void handleToggleStatus() {
    User selected = tableView.getSelectionModel().getSelectedItem();
    if (selected == null) { showMsg("Vui long chon 1 user.", false); return; }

    // Không cho admin tự block chính mình
    if (selected.id == UserSession.get().id) {
        showMsg("Khong the khoa chinh tai khoan cua ban.", false);
        return;
    }

    boolean newStatus = !selected.status;
    String action = newStatus ? "mo khoa" : "khoa";

    Alert alert = new Alert(Alert.AlertType.CONFIRMATION,
        "Ban co muon " + action + " user \"" + selected.username + "\"?",
        ButtonType.YES, ButtonType.NO);
    alert.setTitle("Xac nhan");
    alert.setHeaderText(null);
    alert.showAndWait().ifPresent(btn -> {
        if (btn == ButtonType.YES) {
            if (userRepo.updateStatus(selected.id, newStatus)) {
                loadData();
                showMsg("Da " + action + " user: " + selected.username, true);
            } else {
                showMsg("Thao tac that bai.", false);
            }
        }
    });
}

```

```
        }
    });
}

@FXML
private void handleRefresh() {
    loadData();
    showMsg("Da tai lai danh sach.", true);
}

private void showMsg(String msg, boolean success) {
    lblMessage.setStyle("-fx-text-fill: " + (success ? "#A6E3A1" : "#F38BA8")
+ ";"
    + "-fx-font-size: 12px; -fx-font-style: italic;");
    lblMessage.setText(msg);
}

@FXML
private void goBack() throws IOException {
    SceneSwitcher.switchScene("dashboard.fxml");
}
```

## Ghi Chú Thiết Kế Controllers

| Điểm                                                                   | Giải thích                                                                   |
|------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <code>ProjectService.getInstance()</code>                              | Singleton — tất cả controller dùng chung data, không reload DB khi switch    |
| <code>UserSession.isAdmin()</code>                                     | Kiểm tra role ở 2 nơi: ẩn button trên UI + guard trong handler               |
| <code>btnDelete.setManaged(false)</code> khi ẩn                        | Tránh nút ẩn vẫn chiếm chỗ trong HBox layout                                 |
| <code>loadFromDBAsync(Runnable)</code>                                 | Callback chạy trên UI thread — cập nhật Label/TableView sau khi load xong    |
| <code>Alert CONFIRMATION</code> trước khi xóa/block                    | UX tốt — tránh thao tác destructive nhầm                                     |
| <code>UserListController.initialize()</code> guard redirect            | Nếu URL bị truy cập trực tiếp không qua Dashboard — vẫn bảo vệ được          |
| <code>service.updateStatus(id, status)</code> cập nhật in-memory       | Sau khi update DB, cập nhật luôn object trong map — không cần reload toàn bộ |
| <code>TaskFactory.create()</code> trong <code>AddTaskController</code> | Factory Pattern: Controller không biết new Bug()/new Feature() trực tiếp     |